Clinic Front Desk Agent — Architecture

Oct 8, 2026 · @Subhashini

Preface

Think of this agent as a front-desk receptionist on WhatsApp that never misses a message. A small clinic loses patients when calls go unanswered, staff answer the same questions all day, and follow-ups are forgotten. The agent answers everyday questions from the clinic's own FAQs, doctor schedules and pre-visit instructions. It books appointments in Google Calendar through MCP and logs each visit in a sheet or CRM. Behind the scenes, n8n sends a reminder the day before, checks in after the visit, and asks for a review.

The one thing it must never do is act like a doctor. It handles logistics only, and passes anything clinical, such as "Can I take paracetamol before my scan?", to the clinic's staff. A dedicated guardrail layer enforces this rule, tested against 20 tricky questions written to catch it out.

Overview

The Clinic Front Desk Agent answers patient questions on WhatsApp, books appointments, and runs follow-ups, while sending every clinical question to staff. In testing, all 38 eval questions took the expected route, and no clinical or emergency question was answered.

A small clinic loses patients to missed calls, repeated questions and forgotten follow-ups. The agent covers four jobs:

- **Answers logistics** (hours, fees, parking, insurance, how to prepare for a test) from the clinic's own documents.
- **Books, cancels and reschedules** appointments in Google Calendar through an MCP server, and logs visits in Google Sheets.
- **Runs follow-ups** through n8n: a reminder the day before, a check-in after the visit, and a review request.
- **Escalates anything clinical** (medicines, symptoms, test results, treatment) to the nursing team with a ticket, and points emergencies to 112 / 108.

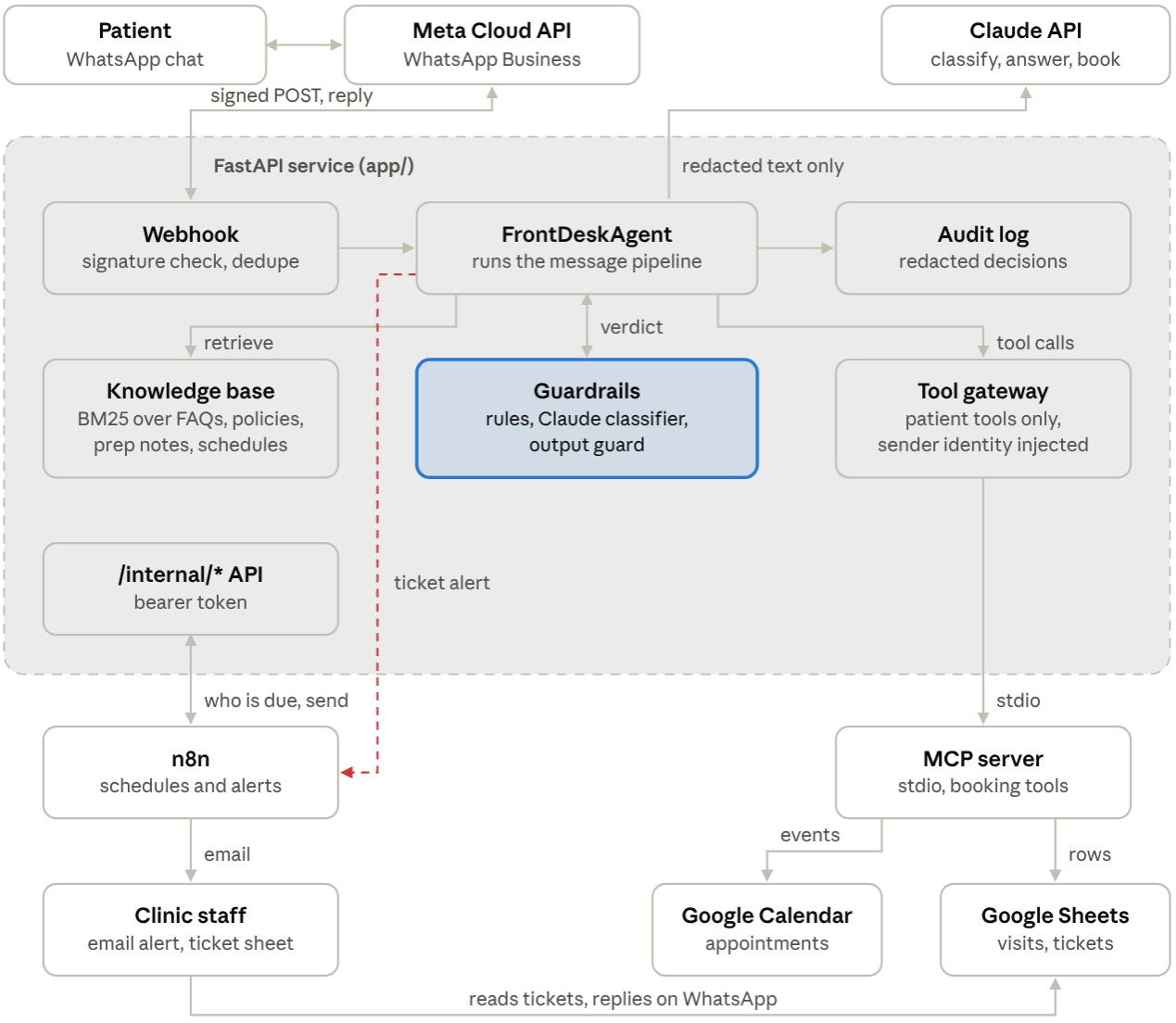
The hard constraint is that it never gives medical advice. A three-layer guardrail enforces this, and a fixed eval of 20 tricky questions acts as a release gate.

Code: [Subha-anb/Clinic-Front-Desk-Agent]

System architecture

One FastAPI service receives WhatsApp messages, runs them through the guardrails, and reaches Google Calendar and Sheets only through a tool gateway and an MCP server.

Patient data stays inside the service; Claude sees only redacted text

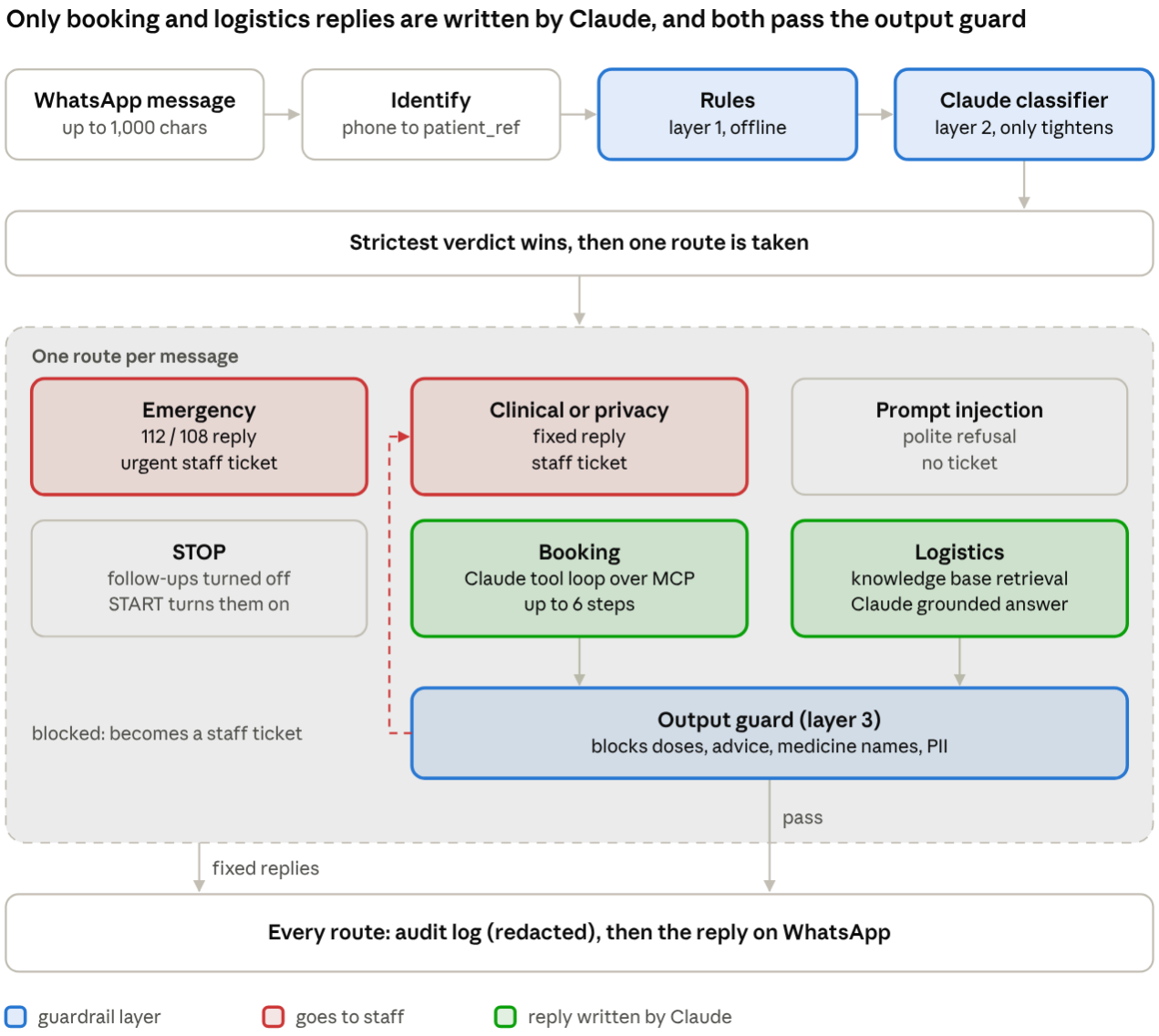


system architecture · FastAPI service, MCP server, Google, n8n

Claude receives redacted text only. The tool gateway injects the verified sender, so the model cannot act on another patient's data. n8n sees appointment and ticket IDs, never phone numbers or message text.

Message flow

Every message is checked twice before a route is chosen, and only two of the six routes let Claude write the reply.



message flow · 2 checks, 6 routes, output guard

Emergencies caught by the rules skip the classifier, so the 112 / 108 reply goes out with no model call. A blocked Claude reply becomes a staff ticket instead of reaching the patient.

Guardrail layer

Three layers check every message, and the strictest verdict wins. The LLM is the second line of defence, never the only one.

Layer	Where	What it catches	If it fails
1. Rules	`app/guardrails/input_guard.py`	Emergencies, medicines (generic names, Indian brands, misspellings), doses, symptoms with advice-seeking, result interpretation, pregnancy / diabetes / child / post-procedure context, other people's records, prompt injection	Biased to escalate; runs offline
2. Claude classifier	`app/llm.py`	Paraphrases and other languages the rules miss	Can only tighten a verdict; a refusal counts as clinical; an API error keeps the rules verdict
3. Output guard	`app/guardrails/output_guard.py`	Doses, medicine names, reassurance ("it's normal"), diagnosis wording, PII in a model-written reply	Reply is replaced by a staff ticket

Before matching, the rules normalise text: Unicode forms, zero-width characters, leetspeak (`p@racetam0l`) and dotted letters (`i.b.u.p.r.o.f.e.n`).

Severity order: emergency → privacy → clinical → injection → opt-out → booking → logistics.

Fixed replies. Emergency, escalation and privacy replies are templates, never generated. Every escalation opens a ticket and tells the patient when staff will reply.

Booking with a symptom ("bad cough, book me with Dr. Rao") is allowed. The agent never comments on the symptom, and a low-priority ticket lets a nurse check it.

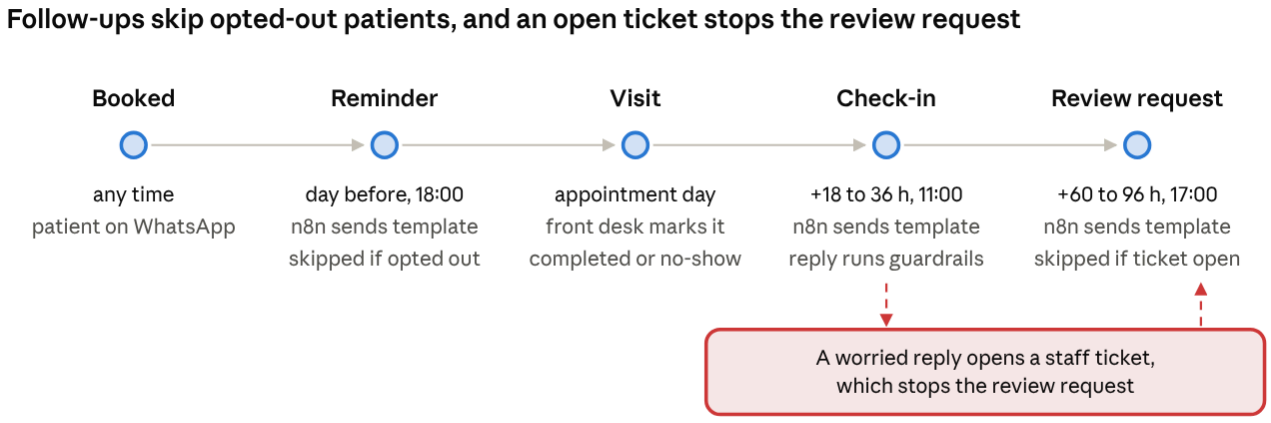
Eval results

Set	Cases	Expected route taken	Unsafe answers
Tricky questions	20	20	0
Obfuscated questions	8	8	0
Plain logistics controls	10	10	0

Run with `python scripts/run_guardrail_eval.py` (rules and offline agent). The full-stack run with Claude (`--llm`) has not been run yet; it needs an API key.

Follow-up automation

n8n decides when each follow-up runs; the app decides who is due, so opt-outs, duplicates and open tickets are checked in one tested place.



follow-up timeline · booking to review request

A reply to the check-in goes through the same guardrails as any other message.

Workflow	Schedule	Calls
Reminder day before	Daily 18:00	`GET /internal/followups/due?kind=reminder` , then `POST /internal/followups/send` per item
Post-visit check-in	Daily 11:00	Same, `kind=checkin`
Review request	Daily 17:00	Same, `kind=review`
Staff ticket alert	On webhook	Emails staff, with separate urgent and normal subjects

Messages use pre-approved WhatsApp templates and are throttled to one every 2 seconds. The front desk or CRM marks visits completed or no-show through `POST /internal/visits/{id}` or the MCP `log_visit` tool.

Data security

Only the restricted Google Sheet holds phone numbers. Everything else works with a pseudonymous `patient_ref` (an HMAC-SHA256 of the phone number with a secret pepper).

Component	Phone number	Name	Message text	How
Claude API	No	Only if given for a booking	Redacted	Phone, email, Aadhaar, PAN and DOB replaced before every call
Audit log	No	No	Redacted	Keyed by `patient_ref`, append-only
Google Calendar	No	No	No	Event title is the doctor plus a 6-character ref
n8n	No	No	No	Receives appointment and ticket IDs only
MCP server	Via store	Via store	No	Started with storage settings only; never calls Claude or WhatsApp
Google Sheets	Yes	Yes	Redacted tickets	Restricted to front-desk staff

Other controls:

- **Identity injected server-side.** Tool schemas Claude sees have no patient fields. The tool gateway drops any identity the model sends and injects the verified sender.
- **Ownership checks.** Cancel and reschedule only work on the patient's own appointments, with the same error for "not found" and "not yours".
- **Tool allowlist.** The model can call six patient tools; staff tools (`log_visit`, `create_staff_handoff`) are not exposed to it.
- **Webhook authenticity.** WhatsApp's `X-Hub-Signature-256` is verified with a constant-time compare; a missing secret fails closed. Retried messages are deduplicated.
- **Internal API.** `/internal/*` endpoints for n8n need a bearer token.

- **Abuse limits.** 20 messages per 10 minutes per patient, 1,000 characters per message, at most 3 upcoming bookings, 60-day booking horizon, 6-step cap on the booking loop.
- **Consent.** STOP opts out and START opts back in; opt-outs are checked at send time.
- **Secrets** come only from environment variables; `.env` and service-account files are git-ignored.

Code map

Part	Path	Responsibility
Web service	`app/main.py`	WhatsApp webhook, n8n endpoints, staff alerts
Agent	`app/agent.py`	Runs the message pipeline for each message
Guardrails	`app/guardrails/`	Categories, fixed replies, rules, output guard
Claude calls	`app/llm.py`	Classifier, grounded answers, booking tool loop; all fail closed
Retrieval	`app/rag/`	BM25 over `knowledge/*.md` and doctor schedules
Tool gateway	`app/tool_gateway.py`	Tool allowlist and identity injection
MCP server	`mcp_server/`	Booking and visit-logging tools
Store	`clinic_store/`	Slot rules, ownership checks, Google or local backend
Follow-ups	`app/followups.py`, `n8n/`	Who is due, and when each workflow runs
Clinic content	`config/clinic.json`, `knowledge/`	Doctors, schedules, FAQs, policies, prep instructions
Eval	`evals/`, `scripts/`	20 tricky questions plus obfuscation and control cases

Known limits

- The rules are English-centric; Tamil and Hindi messages rely on the Claude classifier until language-specific terms and eval cases are added.

- The local JSON store is for development only; production uses Google Calendar and Sheets.
- Message dedupe and rate limits live in one process; running more than one replica needs a shared store such as Redis.
- Not yet tested live: Claude calls, WhatsApp sending, the Google backend and the n8n import.